Supercircuits Package Desciption. Version 1.0

March 14, 2026

1 Introduction

Supercircuit package (supercircuits.py, depends on gencthy.py) is intended for direct simulation of coherent electron and hole transport in semiclassical nanostructures in proximity with superconducting electrodes. It implements the superconducting quantum circuit theory as described in Nazarov and Blanter, *Quantum Transport*, Ch. 2.8. The nanostructure is modeled with a circuit that includes superconducting and, optionally, normal terminals. All superconducting terminals are at the same (zero) voltage to prevent non-stationary dynamics, but may have different phases. The circuit consists of nodes connected to the terminals and to each other. The connectors are generally characterized by transmission distribution. Tunnel, diffusive, ballistic connectors are implemented. Typical output computed consists of electric currents in the terminals at fixed settings of the voltages and all other parameters characterizing the circuit.

The actual use of the package starts with creating an instance of the **SuperCircuit** class. Then the circuit is *built* by adding the terminals, nodes, and connectors. It is so that each element of the circuit should be given a name, and addressed by this name. If you do not like giving names, make your own script to do this for you.

The computation for a circuit with nodes involves solving the quantum circuit theory equations with an iterative method. This may take time and result in non-convergent results and even overflow errors. See Section **Solving hints** for remedies.

2 Building a circuit

Import **SuperCircuit** from the package supercircuits. The instance of the class, our circuit, is created by calling

SuperCircuit (energy =0.0, temperature=0.01)
This returns the class instance. Optional arguments set initial energy and global temperature of the circuit.

The following member functions of the instance add the elements of the setup without connecting them:

addSuperTerminal(name, Delta=1.0, phase=0.0, temperature=None, Gamma=1e-4)
Adds a superconducting terminal with a given name. The optional parameters set superconducting gap Δ, its phase, the terminal temperature (if different from global temperature). The parameter Γ, so-called Dynes parameter, rounds of the singularity in density of states. This parameter may be important for convergence of integration routines.

addNormalTerminal(name, voltage=0.0, temperature=None):
Adds a normal terminal of given name characterized by voltage and temperature (if differs from global temperature).

addNode(name, G=None):
Adds a node of given name. The optional argument sets initial Keldysh Green function of the node (usually not important, since it is recalculated)

check():
This checks the possible errors in the setup revealing disconnected nodes and terminals. There is a predefined "Leakage" terminal that should be disconnected, ignore the warning.

The following functions add the connectors.

addConnector(name,code,contype,Hname,Tname,conductance=1.0, U=None,V=None,transmissions=None):
This adds a connector of a given name. The possible values of **code** are 'TT' (connects two terminals), 'NT'(connects a node and a terminal), 'NN' (connects two nodes). The arguments **Hname**, **Tname** are the names of these nodes/terminals that must already be added to the circuit. The argument **contype** determines the type of the connector. The possible types are 'Tunnel', 'Diffusive', 'Ballistic', and 'Arbitrary'. If the type is 'Arbitrary', one should supply a list of transmission eigenvalues with optional argument **transmissions**. The optional argument **conductance** sets the conductance of the connector. The optional arguments **U** and **V** are of no use for the current version of the package.

addLeakageConnector (name, node, double eth)
Adds a "leakage" connector of a given name to a given node. Such "leakage" connectors account for the decoherence of electrons and holes at finite energy coming from a finite size of the node. The parameter **eth** (Thouless energy) is inversely proportional to the volume of the node, so larger values of Thouless energy correspond to smaller structures.

3 Setting and changing the parameters

setEnergy (double en):
Sets the energy of all terminals to **en**. While used internally in result-giving functions, usually does not have to be called explicitly, unless one wants to

inspect the nodes at a given energy.

setGlobalTemperature(double te):
Sets the global temperature of the setup. Note that if the temperature of some terminals has been previously set to the value that differs from global temperature, the function call does not change their temperatures.

setSuperTerminal (name, Delta=None, Gamma=None, temperature=None, phase=None)
Sets/changes the parameters of the named superconducting terminal. If a parameter is given the value **None** (which happens by default), its value does not change.

setNormalTerminal (self, name, temperature=None, voltage=None):
Sets/changes the parameters of the named normal terminal. If a parameter is given the value **None** (which happens by default), its value does not change.

4 Getting the results

The following functions return the results. Note: all currents computed are to corresponding terminals.

fastDifferentialConductances (name):
This function returns a list of variations of the currents in all terminals upon a change of voltage in the named terminal *assuming* zero temperature of the terminal. It is fast since the circuit is solved only at two energies. It is useful for quick estimations of current-voltage characteristics of the circuit at low temperatures. To recall the order of terminals in the list, print ClassInstance.Terminals.values() that returns the names of the terminals in proper order (disregard "Leakage" terminal that appeares in the list)

FullCurrents (method='trans',scale=1.0,numpoints=200,minen=0,maxen=5.0,acc=1e-2)
This is the main function to use. It returns the numpy array of total currents to each terminal at given settings.

The function performs the integration of current densities over the energies and typically solves the circuit several hundred times at different energies. So do not expect it to be fast.

Three integration methods are implemented in this version.

- 'trans' This is a default method. It integrates by trapezoidal rule using a transformation projecting energies in the interval $(0, \infty)$ to a finite interval. The argument **numpoints** sets the number of points where current densities are computed. The argument **scale** should be of the order of relevant energies (gaps and voltages) in the circuit. Set **minen** to a small positive number in case of convergence complains at precisely zero energy.

- 'straight' This is even a simpler method. It integrates by trapezoidal rule evaluating the current densities at equally spaced energies in the interval

(minen,maxen) to a finite interval. The argument **numpoints** sets the number of evaluations.

- 'adaptive' integrates the densities by an adaptive method from scipy.integrate. As a matter of fact, in this version it is usually slower than more primitive methods and may be prone to convergence problems. The relative accuracy is set by the argument **acc**. Note low default value of this argument.

inspectNode(name)
This returns a tuple of parameters characterizing the state of the node as computed from Keldysh matrix voltage of the node at given energy. These parameters are:

- density of states (relative to the density of states in normal metal)
- pseudophase a complex number, its real part gives a phase of superconductivity induced in the node.
- electron filling factor
- hole filling factor

N.B.Before inspecting the nodes at certain energy, one must use setEnergy() to set the energy and solve() to find the matrix voltages at the nodes.

5 Addressing functions

These functions are not supposed to be used but may appear useful for more involved applications.

terminal(name)
Returns the object associated with the named terminal. One may inspect the parameters like voltage, temperature etc. **Do not try to change** them, use setting functions instead! One can also assign custom properties.

node(name)
The same for the named node.

connector(name)
The same for the named connector.

6 Solving tips

The heart of the class is the function **solve**() that solves iteratively the Keldysh matrix voltages in the nodes. The iterative process and reports about it are controlled by the parameters in ClassInstance.solvepars. You may change these parameters in case of problems with convergence.

- **solvepars.verbose** Integer number defining the level of reporting. 0 - no reports whatever happens. 1 - reports if the desired accuracy is not achieved after maximum number of iterations. 2 - report the results of each iteration cycle. 3 - reports the results of *each* iteration. Default is 1.
- **solvepars.maxiter** Maximum number of iterations per cycle. Increase if problems with convergence.
- **solvepars.goal** Accuracy to achieve in the iteration cycle. Decrease if no time to wait for results of better accuracy.

If the iterations result in overflow errors (they are not caught, so execution stops) for some parameters, the iterative procedure may be too aggressive. This is controlled by property adjust for each node. Use the function **changeAdjust**(newvalue) to set it to lower values in the interval (0,1). The default value is 0.5. Naturally, the calculations will take more time.

A less intuitive method to achieve the convergence is to split a node or several nodes. In this case, a node is replaced by two that are connected by a connector with a conductance larger than all other conductances of the setup. This does not affect much the results provided the conductance is sufficiently large (say, a factor of 20-50 bigger than other conductances) but may drastically improve the convergence.

Check the example file *SuperExamples.py* for examples of building and use. Happy computing!